

Course Name:	Artificial Intelligence Laboratory	Semester:	VI
Date of Performance:	09/04/2026	DIV/ Batch No:	C3
Student Name:	Shreyans Tatiya	Roll No:	16010123325

Experiment No: 7

Title: Solving planning problem using STRIPS or PDDL tools.

Aim: To write STRIPS scripts to solve planning problem and implement them using PDDL tools

Objectives of the Experiment:

By the end of this experiment, students will be able to:

1. Understand the difference between search-based and planning-based problem solving approaches.
2. Model real-world problems using PDDL (Planning Domain Definition Language).
3. Analyze and differentiate between Total Order Planning (TOP) and Partial Order Planning (POP).

COs to be achieved:

CO2: Analyze and solve problems for goal based agent architecture (searching and planning algorithms).

Theory:

Planning is an important area in **Artificial Intelligence** where an intelligent agent determines a sequence of actions required to achieve a specific goal from an initial state. Unlike traditional search techniques, planning uses **logical representations of actions and states** to efficiently reach the desired goal.

One commonly used representation for planning problems is **STRIPS (Stanford Research Institute Problem Solver)**. STRIPS represents actions using three main components:

- **Preconditions** – Conditions that must be true before performing an action.
- **Add list** – Facts that become true after performing the action.

- **Delete list** – Facts that become false after performing the action.

Planning problems are often modeled using **PDDL (Planning Domain Definition Language)**, which allows the specification of:

- The **domain** (types of objects, predicates, and actions)
- The **problem** (initial state and goal state)

Example: Smart Parking Management Problem

Consider a **Smart Parking Management environment** where the goal is to place a **Car in a free Parking Slot**.

1. Searching Approach

A search algorithm treats every parking lot configuration as a separate **state**.

Process

- The algorithm observes the current occupancy of parking slots.
- It generates all possible moves such as:
 - Assign Car1 to Slot1
 - Reserve Slot2 for Car2
 - Vacate Slot3 for a new Car

Limitation

The search algorithm does not understand the **relationships between cars and slots**. It simply explores possible states until it finds the goal state.

If the number of cars and slots increases, the **state space grows exponentially**, making the search inefficient.

2. Planning Approach

A planning algorithm analyzes the **logical structure of the state**.

Example predicates:

- `At(Car1, Entry)`
- `At(Car2, Entry)`
- `Free(Slot1)`
- `Free(Slot2)`

Process

The planner starts from the **goal state** and reasons backward.

Goal:

Occupied(Slot1, Car1)

To achieve this goal:

- **Slot1 must be free**
- **Car1 must be at the Entry**

If another car (for example **Car3**) is already parked in Slot1, the planner deduces that **Car3 must be moved out first**.

Advantages

- Eliminates irrelevant actions
- Reduces search space
- Produces efficient action sequences

Activity

Chosen Planning Problem:

Robot Vacuum Cleaning Two Rooms: A robot vacuum must clean two rooms (Room A and Room B). Initially, both rooms are dirty, and the robot is in Room A. The goal is to clean both rooms.

PDDL Script for solving problem:

Domain Definition:

```
(define (domain vacuum-world)

(:predicates
  (at ?room)
  (clean ?room)
  (dirty ?room)
)

(:action clean-room
  :parameters (?room)
```

```
:precondition (and (at ?room) (dirty ?room))
:effect (and (clean ?room) (not (dirty ?room)))
)

(:action move
  :parameters (?from ?to)
  :precondition (at ?from)
  :effect (and (not (at ?from)) (at ?to))
)

)
```

Problem Definition:

```
(define (problem clean-two-rooms)
  (:domain vacuum-world)

  (:init
    (at RoomA)
    (dirty RoomA)
    (dirty RoomB)
  )

  (:goal
    (and
      (clean RoomA)
      (clean RoomB)
    )
  )

)
```

Explanation of PDDL Model:

The PDDL model defines the environment using predicates, actions, initial state, and goal.

Predicates:

(at ?room) represents the current location of the robot.

(dirty ?room) indicates that a room is dirty.

(clean ?room) indicates that a room is clean.

Actions:

The clean-room action allows the robot to clean a room only if it is present in that room and the room is dirty. After execution, the room becomes clean.

The move action allows the robot to move from one room to another, updating its position.

Initial State:

The robot starts in Room A, and both Room A and Room B are dirty.

Goal:

Both Room A and Room B must be clean.

Working:

The planner generates an efficient sequence of actions:

First, the robot cleans Room A.

Then it moves to Room B.

Finally, it cleans Room B.

Thus, the goal is achieved efficiently without unnecessary steps.

Post Lab Subjective/Objective type Questions:

Question: Answer in brief

1. How does ADL (Action Description Language) extend STRIPS?

Ans:

STRIPS allows only simple representations of actions where preconditions are expressed as conjunctions of positive literals and effects are limited to adding or deleting facts. It cannot represent complex logical conditions.

ADL extends STRIPS by introducing greater expressiveness. It allows negative preconditions, disjunctions, and quantified expressions such as “for all” and “there exists.” It also supports conditional effects, where an effect occurs only if certain conditions are satisfied. Because of these features, ADL can model more realistic and complex planning problems compared to STRIPS.

2. Define Partial Order Planning (POP) and Total Order Planning (TOP). How do they differ?

Ans:

Total Order Planning is a planning approach in which all actions are arranged in a strict linear sequence. Every action has a fixed position, and execution must follow that exact order.

Partial Order Planning is a flexible planning approach in which actions are only ordered when necessary. Independent actions are not constrained and can be executed in any order or even in parallel.

The key difference lies in flexibility. Total Order Planning enforces a complete sequence of actions, whereas Partial Order Planning specifies only the required ordering constraints. As a result, POP is generally more efficient and adaptable, while TOP is simpler but less flexible.

3. Would Partial Order Planning be beneficial in this problem? Why or why not?

Ans:

Partial Order Planning would not be particularly beneficial in this problem. The robot vacuum can only be in one room at a time, and cleaning a room requires the robot to be physically present in that room. This creates a strict dependency between actions.

For example, the robot must clean Room A before moving to Room B, and it must move to Room B before cleaning it. Since these actions cannot be performed independently or in parallel, the plan naturally becomes sequential. Therefore, Total Order Planning is sufficient and more appropriate for this problem.

4. Describe a situation where causal links would be needed in Partial Order Planning for this problem.

Ans:

A causal link is required when one action produces a condition that is necessary for another action. In this problem, cleaning Room B requires the robot to be present in Room B.

For instance, the action of moving from Room A to Room B establishes the condition that the robot is in Room B. This condition is then required for the action of cleaning Room B. Without maintaining this dependency, the planner might attempt to clean Room B without first moving there, which would be incorrect.

5. What are causal links in Partial Order Planning (POP)? Why are they important?

Ans:

Causal links in Partial Order Planning represent dependencies between actions. A causal link connects two actions such that one action establishes a condition that is required by another action.

They are important because they ensure that necessary conditions are preserved throughout the plan. They help maintain the logical correctness of the plan by preventing actions from interfering with required conditions. Without causal links, the planner might generate invalid sequences where actions are executed without satisfying their preconditions.

Conclusion:



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77
(Somaiya Vidyavihar University)

Department of Computer Engineering



This experiment demonstrated how planning techniques improve efficiency by using logical reasoning instead of brute-force search. It also highlighted the importance of structured representations like PDDL in generating optimal and valid action sequences.